V2 Research-to-Live Trading System

Architecture and Implementation Standard

Final Source-of-Truth Runbook for Side-Specialized MLDP/AFML Alpha Edges

Architectural law

If an edge cannot reproduce its exact selected model-input row live using its own runtime package and the same code path used in research, it is not deployable, no matter how attractive the backtest, OOF metrics, or threshold sweep looks.

Field	Value
Document version	1.2 FINAL SOURCE-OF-TRUTH ARCHITECTURE
Date	2026-05-18
Prepared for	Favour Bassey / WELLSMART Trading Models V2
System scope	Multi-asset, multi-alpha, side-specialized research-to-live trading system
Architecture mode	Contract-driven shared-core + edge-runtime architecture
Deployment law	No deployability without runtime replay of selected rows, probability, calibration, AFML, and threshold decision

Purpose of this document

This is the final implementation standard for the V2 rebuild. It is written so future Favour, a senior developer, or another AI can implement, audit, migrate, and extend the system end-to-end without hidden chat context, manual runtime patching, stale scaffold artifacts, or scattered one-off scripts.

Table of Contents

- 1. Executive Summary and Final Architecture Decision
- 2. Blueprint Alignment and Non-Negotiable Contracts
- 3. What Went Wrong and the Permanent Fix
- 4. Professional Repository Architecture
- 5. Shared Core vs Edge-Local Ownership
- 6. Standard Asset Package
- 7. Standard Edge Package
- 8. Exact edge.yaml Schema
- 9. Lifecycle Status Machine
- 10. Runtime.py Design From Day One
- 11. Runtime.py Reference Template
- 12. EdgeScorer API and Scoring Contract
- 13. Standard Edge Lifecycle: 00 to 09
- 14. Detailed Script and Artifact Contracts
- 15. Artifact Naming and Folder Contract
- 16. Feature Schema, Model Input Schema, and Selected-Feature Locking
- 17. Event, Label, Calibration, and Runtime Contract Schemas
- 18. Model Training, CPCV, OOF, Calibration, AFML, and Thresholding
- 19. Replay Audit Tolerances and Deployability Gates
- 20. Edge Manifest Schema
- 21. Portfolio Allocator V2
- 22. Live Manifest Schema
- 23. Dashboard V2
- 24. Live Approved Signal Schema
- 25. Execution Bot V2
- 26. Testing, Audits, Run IDs, and Fail-Closed Rules
- 27. Standard Shared Modules to Build
- 28. Edge Generator and Future No-Assistance Workflow

- 29. Migration Plan for Current Edges
- 30. How to Add a New Edge
- 31. How to Add a New Asset
- 32. Architecture Decision Record Process
- 33. What Must Never Be Done Again
- 34. Implementation Roadmap
- 35. Acceptance Criteria
- 36. AI/Developer Handoff Instructions
- 37. Standard Command Runbook
- 38. Appendix: Templates and Checklists
- 39. Scalability Must Implementation
- Final Professional Decision

1. Executive Summary and Final Architecture Decision

This document defines the final professional V2 architecture for a multi-asset, multi-alpha, side-specialized MLDP/AFML trading system. It converts the Blueprint into a practical end-to-end implementation standard. The system is contract-driven: each alpha edge is a self-contained runtime package; research and live call the same runtime functions; selected features are locked through deterministic contracts; scoring is performed by a shared scoring adapter; the allocator consumes only deployability-PASS manifests; the dashboard only orchestrates; and the execution bot only consumes approved signals.

The design intentionally rejects copying scripts, manually editing artifacts, patching runtime.py after training, or relying on an AI to repair every new edge. Any repeated operation belongs in shared code, a generator, or a thin standardized edge wrapper. Edge-specific code is limited to alpha logic: optional feature additions, candidate events, primary-side mapping, and model-row construction.

Old scattered pattern	Professional V2 replacement
Patch runtime.py after training	runtime.py includes score() from day one and calls shared EdgeScorer; it fails closed until artifacts exist.
Manually copy selected features into selected_features.json	Step 04B locks Step 01 output into canonical selected_features.json and model_input_schema.json using shared logic.
Each edge has a slightly different Step 02/03/04	Step 02/03/04 call shared training and calibration implementations; edge wrappers pass identity and paths only.
Dashboard recreates features	Dashboard calls runtime.build_features, runtime.generate_candidate_events, runtime.build_model_rows, and runtime.score.
Allocator reads folders/CSVs	Allocator consumes only edge_manifest.json with deployment_status=PASS.
Execution bot scores models or builds features	Execution bot validates approved signals and applies risk sizing only.

Final architecture decision

The best professional design is a shared-core plus edge-runtime architecture. Shared modules own reusable infrastructure. Edge packages own side-specialized alpha logic. runtime.py includes score() from scaffold stage and calls shared EdgeScorer. Step 06 verifies runtime scoring readiness; it does not text-patch runtime.py.

2. Blueprint Alignment and Non-Negotiable Contracts

The Blueprint is contract-strict and implementation-flexible. Script names, file splits, and class/function organization may evolve, but behavior must not drift. The non-negotiable parts are side-specialized edges, live-reproducible features, runtime API, schemas, calibrated-probability thresholds, AFML sizing, deployability audits, allocator manifests, dashboard orchestration, and execution signal contracts.

Contract	Required behavior	Fail condition
Side specialization	Each deployable edge is one asset, one family, one side.	A single model mixes long and short candidate distributions.
Runtime source of truth	Each edge owns runtime.py; research and live call the same functions.	Research builds features differently from live.
Bar contract	Use standard_quote_tick bars; threshold_unit=ticks; Proxy_Value only backward-compatible tick count.	Selected features depend on unavailable volume/proxy/tick-speed fields.
Feature contract	Every selected feature is declared, live_available=true, allowed_for_model_input=true, and uses_future_information=false.	Any selected feature is missing, future-derived, or live-incompatible.
Model input contract	Feature order is locked in model_input_schema.json and reproduced exactly at scoring.	Model score uses unordered columns or an implicit feature list.
Probability flow	raw_probability -> sigmoid calibrated_probability -> AFML bet size -> calibrated-threshold decision.	Final deployability threshold is selected on raw_probability.

Contract	Required behavior	Fail condition
Allocator contract	Allocator consumes only PASS edge manifests and representative ledgers.	Allocator reads arbitrary research outputs or non-PASS edges.
Dashboard contract	Dashboard orchestrates runtimes and audits; it never engineers edge-specific features.	Dashboard creates edge-specific features manually.
Execution contract	Execution bot consumes approved signals and live manifest only.	Execution bot builds research features or computes model probabilities.
Fail-closed rule	Broken contract blocks deployability or signal export.	Silent fallback continues with missing features/artifacts.

3. What Went Wrong and the Permanent Fix

The rebuild exposed the exact issue V2 is meant to solve: profitable or promising alpha research can become operationally fragile when implementation grows by one-off edge scripts and manual patches. The problem is not the alpha logic. The problem is hidden coupling, stale scaffold artifacts, copied scripts with hard-coded assumptions, and inconsistent repeated steps.

Pain point	Why it is dangerous	Permanent fix
Stale scaffold artifacts	Step 05 may read empty artifacts/selected_features.json while Step 01 output lives under feature_surgery.	Step 04B deterministically locks selected features and model input schema.
Patch scripts	Copied patchers can contain hard-coded trend_long paths, row builders, or event assumptions.	Runtime template includes score() from day one; Step 06 verifies readiness.
Duplicated training scripts	Tiny script differences silently change internal model logic across edges.	Shared CPCV/training modules; edge wrappers only pass identity and paths.
Manual AI fixes	Future you cannot reproduce the pipeline without chat history.	Runbooks, shared tools, schemas, manifests, and generators become the source of truth.
Dashboard feature logic	Live feature mismatch and metadata collisions appear as new edges are added.	Dashboard imports runtimes and never computes edge features manually.

4. Professional Repository Architecture

The repository must make ownership obvious. Shared infrastructure lives under shared/. Asset data and asset-level state live under assets/<ASSET>/. Edge-specific alpha logic lives inside each edge package. Portfolio allocation, live dashboard, execution, tests, and documentation are separate.

```
trading_models_v2/
  README.md
  pyproject.toml
  config/
    global.yaml
    edge_universe.json
    broker_symbols.yaml
    asset_specs.yaml
    prop_challenge.yaml
  shared/
    bars/
    features/
    ffd/
    regime/
    labeling/
    models/
    calibration/
    scoring/
    validation/
    schemas/
    io/
```

```
    risk/
    edges/
    pipeline/
  assets/
    XAUUSD/
      raw/
      bars/
        standard_quote_tick/
        ffd/
        regime/
      edges/
        trend_long/
        trend_short/
        absorption_long/
        absorption_short/
    BTCUSD/
    USTECH/
  portfolio_allocator/
    scripts/
    schemas/
    outputs/
  portfolio_live/
    dashboard.py
    execution_bot.py
    live_manifest.json
    live_state/
    live_bars/
    outputs/
    audits/
  tests/
    unit/
    integration/
    parity/
    live_dry_run/
  docs/
    architecture/
    runbooks/
    schemas/
    decisions/
```

5. Shared Core vs Edge-Local Ownership

Repeated logic belongs in shared code. Edge-specific alpha assumptions belong in the edge package. Thin edge-local wrappers are allowed because they make the command flow clear, but the internal implementation should call shared utilities whenever possible.

Layer	Owns	Must not own
shared/	Reusable bars, schemas, feature validation, artifact hashing, CPCV, calibration, AFML, scoring, manifest validation, run_id/audit helpers.	Hard-coded edge-specific alpha assumptions unless explicitly parameterized.
edge runtime.py	Side-specialized build_features, generate_candidate_events, build_model_rows, and score wrapper.	Duplicated generic scoring/calibration logic.
edge research/*.py	Ordered wrappers for 00-09 lifecycle.	Unique training/calibration implementations unless formally approved by ADR.
portfolio_allocator/	Edge manifest ingestion, candidate bank, HRP, scenario selection, live manifest export.	Feature engineering or model training.
portfolio_live/dashboard.py	MT5 connection, live bars, runtime orchestration, audits, approved-signal export.	Manual edge feature reconstruction or threshold selection.
execution_bot.py	Signal validation, lot sizing, risk gates, order placement.	Research scoring, feature engineering, or model probability decisions.

6. Standard Asset Package

An asset package owns raw data, forged bars, FFD state, regime state, and edge packages. Asset-level artifacts are shared by multiple edges but must be validated by manifests. An edge must not silently use an asset-level state bundle unless that state bundle has passed contract verification.

Asset-level component	Purpose	Verifier
standard_quote_tick bars	Canonical research/live-compatible bars.	verify_asset_bar_manifest.py
FFD features/state	Frozen fractional differentiation state used by edges.	verify_asset_ffd_manifest.py
HMM/regime features/state	Rolling regime features and live bundle.	verify_asset_hmm_regime_manifest.py
asset_specs.yaml	Point value, tick size, symbol mapping, bar settings.	asset config validator
edge folders	Side-specialized alpha packages.	edge registry and edge skeleton verifier

7. Standard Edge Package

Every edge package must have the same shape. The edge folder name is the side-specialized identity: trend_long, trend_short, absorption_long, absorption_short, reversal_long, etc.

```
assets/XAUUSD/edges/absorption_long/
  edge.yaml
  runtime.py
  feature_expansion.py              # optional edge-local helper
  regime_features.py                # optional wrapper around shared regime features
  candidate_events.py               # edge-specific event generator
  primary_side.py                   # side mapping/filter
  model_rows.py                     # model row and label builder
  contracts/
    bar_contract.json
    feature_schema.json
    event_schema.json
    label_schema.json
    model_input_schema.json
    calibration_schema.json
    live_runtime_contract.json
  configs/
    research.yaml
    labeling.yaml
    model.yaml
    calibration.yaml
  artifacts/
    selected_features.json
    feature_surgery/
    model/
    oof/
    calibration/
    thresholds/
    edge_manifest.json
  research/
    00_validate_edge_contract.py
    01_feature_surgery_clustered_mda.py
    02_train_meta_model_optuna.py
    03_generate_oof_predictions.py
    03b_oof_threshold_sweep.py
    04_fit_sigmoid_calibrator_and_thresholds.py
    04b_lock_selected_features_and_model_input_schema.py
    05_training_artifact_verifier.py
    06_verify_runtime_scoring_ready.py
    07_runtime_score_smoke_test.py
    08_probability_calibration_afml_replay_audit.py
    09_export_edge_manifest.py
  reports/
```

Naming item	Rule	Example
edge_id	<ASSET>/<edge_folder_name>	XAUUSD/absorption_long
edge_stem	<asset_lower>_<edge_folder_name>	xauusd_absorption_long
side_name	long or short	long

Naming item	Rule	Example
side_value	+1 or -1	+1
model	<edge_stem>_meta_model.pkl	xauusd_absorption_long_meta_model.pkl
calibrator	probability_calibrator_sigmoid_side_<side>.json	probability_calibrator_sigmoid_side_long.json
selected threshold	<edge_stem>_selected_threshold.json	xauusd_absorption_long_selected_threshold.json

8. Exact edge.yaml Schema

edge.yaml is the edge identity and configuration source used by generators, verifiers, runtime contract checks, and edge registry tooling. It prevents future wrappers from guessing asset/family/side/labeling assumptions from folder names alone. Folder names remain the default identity, but edge.yaml must confirm them.

```
schema_version: v2.1
edge_id: XAUUSD/absorption_long
asset: XAUUSD
family: absorption
side_name: long
side_value: 1
edge_stem: xauusd_absorption_long
status: scaffold_not_deployable   # status machine values defined in Section 9
runtime_py: runtime.py

bar_contract:
  bar_type: standard_quote_tick
  threshold_unit: ticks
  proxy_value_policy: backward_compatible_tick_count_only

alpha_contract:
  candidate_event_generator: absorption_long_v1_downside_failed_displacement_candidate_events
  primary_side_rule: Absorption_Event_Type=-1 -> side=+1 long
  edge_specific_features_allowed: true
  requires_ffd: true
  requires_regime: optional

labeling:
  row_builder: absorption_long_v1_1r_triple_barrier_model_rows
  vertical_horizon: 100
  pt_multiplier: 1.0
  sl_multiplier: 1.0
  tie_policy: sl_first
  tick_resolution_price: mid
  long_barrier_contract: pt_price_above_entry_sl_price_below_entry
  short_barrier_contract: pt_price_below_entry_sl_price_above_entry

research:
  cv: combinatorial_purged_cv
  n_folds: 5
  n_test_folds: 2
  manual_embargo_size: 100
  weights: class_weight_mode_x_event_vol_x_average_uniqueness

probability:
  flow: raw_probability -> sigmoid calibrated_probability -> AFML bet size -> calibrated-threshold decision
  final_edge_threshold_space: calibrated_probability
  final_live_threshold_source: allocator_live_manifest

runtime_contract:
  required_functions:
    - load_contract
    - validate_environment
    - build_features
    - generate_candidate_events
    - build_model_rows
    - load_model
    - load_calibrator
    - score
    - explain_missing_features
```

edge.yaml field	Required?	Purpose

edge.yaml field	Required?	Purpose
edge_id / asset / family / side	Yes	Locks identity and side specialization.
status	Yes	Controls lifecycle and fail-closed behavior.
bar_contract	Yes	Prevents asset/edge from silently using incompatible bars.
alpha_contract	Yes	Documents what makes the edge unique.
labeling	Yes	Locks horizon, PT/SL, and tie policy.
research	Yes	Locks model training and validation method.
probability	Yes	Locks probability and threshold semantics.
runtime_contract	Yes	Defines required runtime API.

9. Lifecycle Status Machine

Every edge must move through explicit states. The status must be stored in edge_universe.json and/or edge.yaml. Scripts may update status only when their required outputs exist and their verifier passes. No dashboard, allocator, or execution component may infer deployability from folder presence.

Status	Entered when	Allowed next statuses
scaffold_not_deployable	Edge skeleton exists but alpha logic/artifacts are incomplete.	runtime_research_ready
runtime_research_ready	candidate_events.py, primary_side.py, model_rows.py, and runtime research self-test pass.	research_dataset_exported
research_dataset_exported	Feature matrix, candidate events, and model rows exported from runtime-built rows.	feature_surgery_complete
feature_surgery_complete	Step 01 selected features and manifest exist.	model_trained_not_calibrated
model_trained_not_calibrated	Step 02 model and training manifest exist.	oof_generated
oof_generated	Step 03 OOF predictions cover required rows.	calibrated_not_verified
calibrated_not_verified	Step 04 calibrator, calibrated OOF, threshold sweep, and selected threshold exist.	features_locked
features_locked	Step 04B canonical selected_features.json and model_input_schema.json locked.	training_artifacts_pass
training_artifacts_pass	Step 05 PASS report exists.	runtime_scoring_ready
runtime_scoring_ready	Step 06 confirms runtime.score works through shared EdgeScorer.	smoke_test_pass
smoke_test_pass	Step 07 latest/sample score smoke test passes.	replay_audit_pass
replay_audit_pass	Step 08 raw/calibrated/AFML/threshold replay passes.	deployability_pass
deployability_pass	Step 09 exported edge_manifest.json with deployment_status=PASS.	allocator_candidate
allocator_candidate	Allocator can consume the PASS manifest.	live_selected or watchlist
live_selected	Allocator live manifest selects the edge.	live_disabled
live_disabled	Risk/operator disables the edge without deleting artifacts.	live_selected

10. Runtime.py Design From Day One

runtime.py is the heart of the system. It is not something to patch at the end. A scaffolded edge must already expose all required runtime API functions, including score(). Before training artifacts exist, score() must fail closed with EdgeScoringNotReady. After Step 05 and Step 06 pass, the same score() function must work without code changes.

Function	Purpose	Research use	Live use
load_contract()	Load identity and schema contracts.	Preflight and audit.	Dashboard startup validation.
validate_environment()	Check bars, FFD/regime, artifact availability, versions.	Before pipeline stages.	Before live scoring.
build_features(bars=None, mode="research/live")	Produce full feature matrix using same code.	Feature surgery/model rows.	Live features.
generate_candidate_events(features, mode="research/live")	Produce side-specialized candidate events.	Candidate event dataset.	Live candidate events.
build_model_rows(features, events, bars=None, mode="research/live")	Create final model rows; labels only in research.	Training/OOF rows.	Unlabeled live scoring rows.
load_model()	Load raw probability model through EdgeScorer or explicit helper.	Replay/smoke tests.	Runtime scoring.
load_calibrator()	Load sigmoid calibrator.	Replay/smoke tests.	Runtime scoring.
score(model_rows=None, latest_only=False)	Order selected features and compute raw/calibrated/AFML/approved.	Smoke/replay.	Dashboard scoring.
explain_missing_features(model_rows)	Structured missing feature report.	Audit failures.	Dashboard fail output.

11. Runtime.py Reference Template

This template is the standard shape new edge scaffolds should generate. Edge-specific implementation lives inside build_features, generate_candidate_events, and build_model_rows. Generic scoring is delegated to shared.scoring.edge_scorer.EdgeScorer.

```
from __future__ import annotations

from pathlib import Path
from typing import Any
import pandas as pd

from shared.scoring.edge_scorer import EdgeScorer, EdgeScoringNotReady
from shared.validation.edge_contracts import RuntimeContractError

EDGE_ROOT = Path(__file__).resolve().parent
EDGE_NAME = EDGE_ROOT.name
ASSET = EDGE_ROOT.parents[1].name
EDGE_ID = f"{ASSET}/{EDGE_NAME}"
EDGE_STEM = f"{ASSET.lower()}_{EDGE_NAME}"


def load_contract() -> dict[str, Any]:
    """Load edge.yaml and contract JSON files. Fail if identity mismatches folder layout."""
    ...


def validate_environment(*, require_scoring_artifacts: bool = False) -> dict[str, Any]:
    """Validate bars, FFD/regime state, schemas, and optional scoring artifacts."""
    ...


def build_features(bars: pd.DataFrame | None = None, *, mode: str = "research", drop_warmup_rows: bool = False) -> pd.DataFrame:
    """Build all declared edge features using causal logic only."""
    ...


def generate_candidate_events(features: pd.DataFrame, *, mode: str = "research") -> pd.DataFrame:
    """Generate side-specialized candidate events. No labels, probabilities, or execution decisions."""
    ...
```

```
def build_model_rows(
    features: pd.DataFrame,
    events: pd.DataFrame,
    bars: pd.DataFrame | None = None,
    *,
    mode: str = "research",
) -> pd.DataFrame:
    """Build model rows. In research mode labels may be added; in live mode labels are forbidden."""
    ...


def load_model():
    return EdgeScorer.from_edge_root(EDGE_ROOT).load_model()


def load_calibrator():
    return EdgeScorer.from_edge_root(EDGE_ROOT).load_calibrator()


def explain_missing_features(model_rows: pd.DataFrame) -> dict[str, Any]:
    return EdgeScorer.from_edge_root(EDGE_ROOT).explain_missing_features(model_rows)


def score(model_rows: pd.DataFrame | None = None, *, latest_only: bool = False) -> pd.DataFrame:
    """Score model rows through shared EdgeScorer. Never text-patch this function after training."""
    if model_rows is None:
        features = build_features(mode="live", drop_warmup_rows=False)
        events = generate_candidate_events(features, mode="live")
        model_rows = build_model_rows(features=features, events=events, mode="live")
    scorer = EdgeScorer.from_edge_root(EDGE_ROOT)
    return scorer.score(model_rows=model_rows, latest_only=latest_only)


def _self_test() -> None:
    """Run feature/event/model-row self-test. Scoring may fail closed before artifacts exist."""
    ...


if __name__ == "__main__":
    _self_test()
```

12. EdgeScorer API and Scoring Contract

The shared EdgeScorer is the professional replacement for runtime patch scripts. It must never know edge alpha logic. It only loads canonical artifacts and scores already-built model rows. Every edge runtime calls the same EdgeScorer API.

API	Return	Required behavior
EdgeScorer.from_edge_root(edge_root: Path)	EdgeScorer	Resolve edge_id, edge_stem, side, artifact paths, and contract paths.
validate_artifacts()	dict	Fail if selected_features, model_input_schema, model, calibrator, or selected threshold are missing/mismatched.
load_selected_features()	list[str]	Read artifacts/selected_features.json and preserve order.
load_model_input_schema()	dict	Confirm contracts/model_input_schema.json matches selected_features.json exactly.
load_model()	model object	Load artifacts/model/<edge_stem>_meta_model.pkl.
load_calibrator()	dict/object	Load probability_calibrator_sigmoid_side_<side>.json and optional pkl.
load_threshold()	dict	Load artifacts/thresholds/<edge_stem>_selected_threshold.json and confirm calibrated_probability space.
explain_missing_features(model_rows)	dict	Return missing selected features, extra label/probability fields, dtype issues, and NaN summary.
score(model_rows, latest_only=False)	pd.DataFrame	Compute raw_probability, calibrated_probability, afml_bet_size, selected_threshold, approved_signal, and audit metadata.

Required score output column	Meaning
edge_id	Edge identity.
score_timestamp	UTC timestamp when score was produced.
raw_probability	Model predict_proba positive-class output.
calibrated_probability	Sigmoid/identity calibrated probability.
afml_bet_size	AFML probability-derived bet size.
selected_threshold	Edge default calibrated threshold, unless live manifest overrides.
threshold_probability_column	Must equal calibrated_probability.
approved_signal	calibrated_probability >= selected_threshold.
model_artifact_hash	Hash used for manifest/replay.
calibrator_artifact_hash	Hash used for manifest/replay.
threshold_artifact_hash	Hash used for manifest/replay.
selected_features_hash	Hash of canonical selected feature list.

Step 06 design decision

Step 06 must be 06_verify_runtime_scoring_ready.py. It should not patch runtime.py. It verifies that runtime.py already exposes score(), that EdgeScorer can load artifacts, and that latest_only scoring produces required columns.

13. Standard Edge Lifecycle: 00 to 09

Step	Name	Owns	Writes	Deployable?
00	validate_edge_contract	Edge identity, scaffold, schemas, required runtime API.	Validation report.	No
01	feature_surgery_clustered_mda	Feature selection from runtime-built research model rows.	artifacts/feature_surgery/*selected_features.json and manifest.	No
02	train_meta_model_optuna	CPCV/MLDP model training with selected features.	model.pkl, best_params, feature_importance, model_training_manifest.	No
03	generate_oof_predictions	CPCV OOF raw_probability.	oof_predictions.csv and oof_predictions_manifest.json.	No
03b	raw_oof_threshold_sweep_diagnostics	Raw-probability diagnostics only.	raw_threshold_sweep.csv and summary.	No
04	fit_sigmoid_calibrator_and_thresholds	Sigmoid calibration, AFML, calibrated threshold sweep.	calibrator, calibrated_oof, selected_threshold.	No
04b	lock_selected_features_and_model_input_schema	Promote Step 01 selected features to canonical contracts.	artifacts/selected_features.json and contracts/model_input_schema.json.	No
05	training_artifact_verifier	Verify model/calibration/threshold/feature contracts are internally consistent.	training_artifact_verification_report.json.	No
06	verify_runtime_scoring_ready	Confirm runtime score is implemented and artifacts load without patching.	runtime_scoring_ready_report.json.	No
07	runtime_score_smoke_test	Score latest row or sample rows through runtime.py::score.	runtime_score_smoke_test_report.json.	No
08	probability_calibration_afml_replay_audit	Replay raw, calibrated, AFML, threshold decisions against research artifacts.	replay audit report.	No
09	export_edge_manifest	Export deployability PASS	artifacts/edge_manifest.json.	Yes, if

Step	Name	Owns	Writes	Deployable?
		manifest only if all gates pass.		PASS

14. Detailed Script and Artifact Contracts

Every pipeline step must have clear inputs, outputs, and fail conditions. A later step must never infer a missing artifact from a different folder unless that behavior is explicitly part of the contract.

Step	Required inputs	Required outputs	Important fail conditions
01	runtime model rows, feature_schema.json	feature_surgery selected features, feature surgery manifest	Selected feature not in schema, future feature selected, no features selected.
02	model_rows.csv, Step 01 selected features, feature_schema.json	model.pkl, best_params, training manifest	Wrong side, wrong edge_id, leakage feature, non-CPCV training, no class diversity.
03	model rows, selected features, best params	OOF predictions and manifest	OOF coverage < required, wrong side, selected-feature missing.
03b	OOF predictions	raw threshold diagnostics	Using as final deployable threshold. This is diagnostic only.
04	OOF predictions and raw diagnostics	calibrator, calibrated OOF, selected calibrated threshold, manifests	Threshold chosen on raw_probability, no calibration report, invalid calibrator.
04b	feature_surgery selected features, feature_schema.json	canonical selected_features.json and model_input_schema.json	Feature not live-safe, mismatch edge_id, duplicate features.
05	all training/calibration/threshold/canonical contracts	verification report	Any mismatch, stale scaffold, wrong counts, unsafe features.
06	runtime.py, selected_features, model_input_schema, model, calibrator, threshold	runtime scoring ready report	runtime score missing or patch needed.
07	runtime score ready	smoke report	Missing score columns, score fails, approved_signal invalid.
08	runtime outputs and research artifacts	replay audit report	raw/calibrated/AFML/threshold mismatch.
09	all PASS reports	edge_manifest.json with deployment_status=PASS	Any prior gate not PASS.

15. Artifact Naming and Folder Contract

Every edge uses the same artifact naming contract. Future scripts must not invent alternative stems or read arbitrary CSVs. If a new artifact is required, add it to this contract or create an ADR.

Artifact	Canonical path pattern	Owner step
Research feature matrix	artifacts/research_dataset/<edge_stem>_feature_matrix.csv	export_research_dataset.py / runtime export
Candidate events	artifacts/research_dataset/<edge_stem>_candidate_events.csv	export_research_dataset.py / runtime export
Model rows	artifacts/research_dataset/<edge_stem>_model_rows.csv	export_research_dataset.py / runtime export
Research dataset manifest	artifacts/research_dataset/research_dataset_manifest.json	export_research_dataset.py
Feature surgery selected	artifacts/feature_surgery/<edge_stem>_selected_features.json	Step 01

Artifact	Canonical path pattern	Owner step
list		
Feature surgery manifest	artifacts/feature_surgery/feature_surgery_manifest.json	Step 01
Canonical selected features	artifacts/selected_features.json	Step 04B
Model input schema	contracts/model_input_schema.json	Step 04B
Model pickle	artifacts/model/<edge_stem>_meta_model.pkl	Step 02
Best params	artifacts/model/<edge_stem>_meta_model_best_params.json	Step 02
Model training manifest	artifacts/model/model_training_manifest.json	Step 02
OOF predictions	artifacts/oof/<edge_stem>_oof_predictions.csv	Step 03
OOF manifest	artifacts/oof/oof_predictions_manifest.json	Step 03
Raw OOF sweep	artifacts/oof/<edge_stem>_oof_raw_threshold_sweep.csv	Step 03B
Calibrator json	artifacts/calibration/probability_calibrator_sigmoid_side_<side>.json	Step 04
Calibrated OOF	artifacts/calibration/<edge_stem>_calibrated_oof_predictions.csv	Step 04
Calibration report	artifacts/calibration/<edge_stem>_calibration_report.json	Step 04
Calibrated threshold sweep	artifacts/thresholds/<edge_stem>_calibrated_threshold_sweep.csv	Step 04
Selected threshold	artifacts/thresholds/<edge_stem>_selected_threshold.json	Step 04
Training verifier report	reports/training_artifact_verification_report.json	Step 05
Runtime scoring ready report	reports/runtime_scoring_ready_report.json	Step 06
Smoke report	reports/runtime_score_smoke_test_report.json	Step 07
Replay audit report	reports/probability_calibration_afml_replay_audit_report.json	Step 08
Edge manifest	artifacts/edge_manifest.json	Step 09

16. Feature Schema, Model Input Schema, and Selected-Feature Locking

The selected-feature problem must be solved by architecture, not manual edits. Step 01 writes research output under feature_surgery. Step 04B promotes that list into canonical artifacts. Step 05, runtime scoring, replay audit, manifest export, allocator, and dashboard consume canonical artifacts only.

Artifact	Owner	Purpose	Mutable?
artifacts/feature_surgery/<edge_stem>_selected_features.json	Step 01	Research selected-feature output.	Created by Step 01.
artifacts/selected_features.json	Step 04B	Canonical selected-feature list and order.	Locked after Step 04B; changes require rerun downstream steps.
contracts/model_input_schema.json	Step 04B	Canonical model input column order for runtime scoring.	Locked after Step 04B.
contracts/feature_schema.json	Edge developer/runtime owner	All produced features, safety flags, formulas.	Updated when build_features/build_model_rows changes.

```
# selected_features.json template
{
  "schema_version": "v2.1",
  "edge_id": "XAUUSD/absorption_long",
  "status": "selected_features_locked_not_deployable",
  "feature_order_locked": true,
  "selected_feature_count": 94,
  "source_artifact":
"assets/XAUUSD/edges/absorption_long/artifacts/feature_surgery/xauusd_absorption_long_selected_features.json",
  "locked_at": "2026-05-18T...Z",
  "selected_features": ["feature_1", "feature_2"]
}

# model_input_schema.json template
{
  "schema_version": "v2.1",
  "edge_id": "XAUUSD/absorption_long",
  "live_must_reproduce_exact_selected_rows": true,
  "model_input_order_locked": true,
  "selected_features_artifact": "artifacts/selected_features.json",
  "model_input_column_count": 94,
  "model_input_columns": ["feature_1", "feature_2"]
}
```

17. Event, Label, Calibration, and Runtime Contract Schemas

Every contract JSON must be explicit enough for a verifier to reject wrong-side rows, missing live features, and future-derived columns. These schemas are not decoration; they are executable contracts.

```
# event_schema.json minimum
{
  "schema_version": "v2.1",
  "edge_id": "XAUUSD/absorption_long",
  "candidate_event_generator": "absorption_long_v1_downside_failed_displacement_candidate_events",
  "side_name": "long",
  "side_value": 1,
  "required_event_columns": ["Datetime", "side", "event_id", "Event_Vol"],
  "forbidden_event_columns": ["meta_label", "raw_probability", "approved_signal"],
  "uses_future_information": false
}

# label_schema.json minimum
{
  "schema_version": "v2.1",
  "edge_id": "XAUUSD/absorption_long",
  "row_builder": "absorption_long_v1_1r_triple_barrier_model_rows",
  "side_value": 1,
  "vertical_horizon": 100,
  "pt_multiplier": 1.0,
  "sl_multiplier": 1.0,
  "tie_policy": "sl_first",
  "long_barrier_contract": "pt_price_above_entry_sl_price_below_entry",
  "live_scoring_must_not_use_label_columns": true
}

# calibration_schema.json minimum
{
  "schema_version": "v2.1",
  "edge_id": "XAUUSD/absorption_long",
  "probability_flow": "raw_probability -> sigmoid calibrated_probability -> AFML bet size -> calibrated-threshold decision",
  "final_edge_threshold_space": "calibrated_probability",
  "forbidden_threshold_space": "raw_probability"
}

# live_runtime_contract.json minimum
{
  "schema_version": "v2.1",
  "edge_id": "XAUUSD/absorption_long",
  "runtime_required_functions": ["load_contract", "validate_environment", "build_features", "generate_candidate_events",
"build_model_rows", "load_model", "load_calibrator", "score", "explain_missing_features"],
  "dashboard_may_call": ["validate_environment", "build_features", "generate_candidate_events", "build_model_rows", "score"],
```

```
  "dashboard_must_not_compute_features": true
}
```

18. Model Training, CPCV, OOF, Calibration, AFML, and Thresholding

The MLDP model design is primary-candidate plus meta-label. Direction is fixed by the side-specialized edge. The meta-model only decides accept/reject for that side. Internal training logic must remain shared or provably identical across edges except for edge-specific data and selected features.

Component	Professional standard
Candidate event	Generated by edge runtime; side is fixed by edge identity.
Meta-label	1 means this side-specific candidate was worth taking; 0 means reject.
CV method	CPCV with t1 overlap purge and embargo for event horizon overlap.
Weights	class_weight_mode x Event_Vol x average uniqueness.
OOF probability	Out-of-fold raw_probability only; never use in-sample probabilities for threshold deployment.
Calibration	Fit sigmoid on OOF predictions, not in-sample training predictions.
AFML bet size	Computed from calibrated_probability, not raw_probability.
Threshold selection	Final edge default threshold selected in calibrated_probability space.
Allocator threshold	Portfolio allocator may override/select final live threshold in live manifest after overlap-aware portfolio simulation.

Shared training rule

Edge-local Step 02 and Step 03 wrappers must not contain a unique CPCV implementation. They should call shared/models/cpcv_training.py. If an edge truly requires different training logic, create an Architecture Decision Record before implementation.

19. Replay Audit Tolerances and Deployability Gates

An edge is not deployable because Step 04 looks promising or Step 05 passes. It becomes deployable only after runtime scoring and replay audit prove that runtime outputs reproduce research probability, calibration, AFML, and threshold decisions exactly enough for live use.

Replay check	Tolerance / requirement
Selected feature set	Exact same feature names as artifacts/selected_features.json.
Selected feature order	Exact same order as contracts/model_input_schema.json.
Missing selected features	0 missing features.
Forbidden label/probability columns in model input	0 forbidden columns used.
Model row overlap	>= configured minimum; default >= 95% of research rows where horizon allows.
raw_probability max diff	<= 1e-12 for same model/library path; <= 1e-9 allowed only with documented numerical/platform reason.
calibrated_probability max diff	<= 1e-12; hard fail above 1e-9.
afml_bet_size max diff	<= 1e-12; hard fail above 1e-9.
threshold mismatches	0 mismatches.
approved_signal mismatches	0 mismatches.
artifact hashes	All required artifacts hashed and recorded in report/manifest.
deployment_status	PASS only if all mandatory gates pass.

Gate	Must prove
Feature parity	Runtime can reproduce selected model-input rows with same selected features and order.
Raw probability replay	Runtime model raw_probability equals research replay within

Gate	Must prove
	tolerance.
Calibration replay	Runtime calibrated_probability equals Step 04 calibrated output.
AFML replay	Runtime afml_bet_size equals Step 04/AFML formula output.
Threshold replay	approved_signal matches calibrated threshold decision.
Artifact hashes	Model, calibrator, selected features, threshold, schemas are hashed into manifest.
Manifest status	deployment_status=PASS only after all mandatory reports PASS.

20. Edge Manifest Schema


```
{
  "schema_version": "v2.1",
  "edge_id": "XAUUSD/absorption_long",
  "created_at": "2026-05-18T...Z",
  "identity": {
    "asset": "XAUUSD",
    "family": "absorption",
    "side_name": "long",
    "side_value": 1,
    "edge_stem": "xauusd_absorption_long"
  },
  "contracts": {
    "edge_yaml": "edge.yaml",
    "feature_schema": "contracts/feature_schema.json",
    "model_input_schema": "contracts/model_input_schema.json",
    "label_schema": "contracts/label_schema.json",
    "calibration_schema": "contracts/calibration_schema.json",
    "live_runtime_contract": "contracts/live_runtime_contract.json"
  },
  "artifacts": {
    "selected_features": "artifacts/selected_features.json",
    "model": "artifacts/model/xauusd_absorption_long_meta_model.pkl",
    "calibrator": "artifacts/calibration/probability_calibrator_sigmoid_side_long.json",
    "selected_threshold": "artifacts/thresholds/xauusd_absorption_long_selected_threshold.json",
    "calibrated_oof": "artifacts/calibration/xauusd_absorption_long_calibrated_oof_predictions.csv"
  },
  "probability_flow": "raw_probability -> sigmoid calibrated_probability -> AFML bet size -> calibrated-threshold decision",
  "threshold": {
    "threshold_probability_column": "calibrated_probability",
    "edge_default_threshold": 0.55,
    "final_live_threshold_source": "allocator/live manifest"
  },
  "runtime": {
    "runtime_py": "runtime.py",
    "required_functions": ["load_contract", "validate_environment", "build_features", "generate_candidate_events",
"build_model_rows", "load_model", "load_calibrator", "score", "explain_missing_features"]
  },
  "audit_reports": {
    "training_artifacts": "reports/training_artifact_verification_report.json",
    "runtime_scoring_ready": "reports/runtime_scoring_ready_report.json",
    "runtime_smoke": "reports/runtime_score_smoke_test_report.json",
    "replay_audit": "reports/probability_calibration_afml_replay_audit_report.json"
  },
  "artifact_hashes": {},
  "deployment_status": "PASS",
  "deployable": true
}
```


21. Portfolio Allocator V2

The allocator is not an edge researcher. It consumes deployable alpha legs and constructs a portfolio. It never reads arbitrary research folders or non-PASS edges. It builds calibrated candidate banks, classifies core/watchlist, evaluates overlap and diversification, runs HRP, stress tests, and exports the live manifest.

Allocator stage	Required behavior
Manifest ingestion	Load only edge_manifest.json where deployment_status=PASS.
Candidate bank	Use calibrated OOF/event ledgers and representative threshold grids.
Core/watchlist classification	Classify edge legs by robustness, trade count, expectancy, drawdown, year stability, correlation, and overlap.
Correlation/overlap analysis	Measure event overlap, return correlation, cluster structure, and uniqueness.
HRP weighting	Use representative return streams, not raw arbitrary PnL curves.
Prop challenge simulation	Apply capital, target, max drawdown, risk caps, direct-weight calibrated-AFML sizing.
Monte Carlo stress	Stress edge sequence, gap, losing streaks, overlap, and drawdown.
Live manifest export	Write selected alpha legs, thresholds, weights, risk caps, symbol mapping, and artifact hashes.

22. Live Manifest Schema

```
{
  "schema_version": "v2.1",
  "created_at": "2026-05-18T...Z",
  "portfolio_id": "xauusd_multi_edge_v2",
  "mode": "demo_only",
  "capital_base": 10000.0,
  "risk_model": "direct_weight_calibrated_afml",
  "global_risk_caps": {
    "max_total_open_risk_fraction": 0.01,
    "daily_loss_soft_stop_fraction": 0.02,
    "challenge_max_drawdown_fraction": 0.08
  },
  "selected_edges": [
    {
      "edge_id": "XAUUSD/absorption_long",
      "asset": "XAUUSD",
      "side_name": "long",
      "side_value": 1,
      "edge_manifest": "assets/XAUUSD/edges/absorption_long/artifacts/edge_manifest.json",
      "edge_manifest_hash": "...",
      "final_live_threshold": 0.55,
      "threshold_probability_column": "calibrated_probability",
      "threshold_source": "allocator_overlap_aware_simulation",
      "portfolio_weight": 0.25,
      "max_edge_risk_fraction": 0.0025,
      "enabled": true
    }
  ]
}
```

23. Dashboard V2

The dashboard is an orchestrator and audit surface. It must not contain alpha logic. Adding a new deployable edge should require no dashboard code changes. The dashboard loads the live manifest, imports listed edge runtimes, builds live bars, calls runtime functions, writes run_id-aware audits, and exports approved signals.

Dashboard operation	Allowed?	Comment
Build standard_quote_tick bars	Yes	Using shared bar builder.
Call runtime.build_features	Yes	Edge owns feature logic.
Call runtime.generate_candidate_events	Yes	Edge owns candidate logic.
Call runtime.build_model_rows	Yes	Edge owns alignment logic.
Call runtime.score	Yes	Shared EdgeScorer through runtime.
Compute edge-specific features manually	No	Forbidden. Fix runtime/schema instead.
Place orders	No	Execution bot owns orders.

Dashboard operation	Allowed?	Comment
Choose thresholds	No	Allocator/live manifest owns thresholds.

24. Live Approved Signal Schema


```
{
  "schema_version": "v2.1",
  "run_id": "20260518T142233Z_dashboard_dry_run_001",
  "signal_id": "XAUUSD_absorption_long_2026-04-07T22:54:43.778Z",
  "edge_id": "XAUUSD/absorption_long",
  "asset": "XAUUSD",
  "broker_symbol": "XAUUSD.x",
  "side_name": "long",
  "side_value": 1,
  "event_timestamp": "2026-04-07T22:54:43.778Z",
  "score_timestamp": "2026-05-18T14:22:33Z",
  "raw_probability": 0.55800921,
  "calibrated_probability": 0.56452041,
  "afml_bet_size": 0.10353547,
  "selected_threshold": 0.55,
  "threshold_probability_column": "calibrated_probability",
  "approved_signal": true,
  "entry_reference_price": 2350.50,
  "live_manifest_hash": "...",
  "edge_manifest_hash": "...",
  "artifact_hashes": {
    "model": "...",
    "calibrator": "...",
    "selected_features": "...",
    "selected_threshold": "..."
  }
}
```


25. Execution Bot V2

The execution bot must be deliberately dumb about research internals. It validates approved signals, deduplicates them, calculates lot size from live manifest risk rules, applies broker constraints and risk caps, then places orders only after every gate passes.

Execution bot gate	Required check
Signal contract	edge_id, asset, side, timestamp, run_id, approved_signal, calibrated_probability, afml_bet_size, threshold, manifest version.
Manifest validation	Signal edge exists in live manifest and artifact hashes match.
Deduplication	No duplicate open signal for same edge/event.
Risk sizing	direct-weight calibrated-AFML sizing from allocator weights and risk caps.
Broker constraints	min lot, max lot, step size, IOC/filling mode, symbol availability.
Risk caps	daily stop, max open risk, per-edge cap, per-asset cap, challenge drawdown.
Order action	Only after all gates PASS.

26. Testing, Audits, Run IDs, and Fail-Closed Rules

Test type	Purpose
Unit tests	Bar close logic, deterministic feature functions, AFML transform, schema parsing, artifact hashing.
Integration tests	Runtime builds features/events/model rows and scoring loads artifacts.
Parity tests	Historical replay produces same selected rows and scores as research artifacts.
Live dry-run tests	Historical tick/live replay without order_send; writes audits and

Test type	Purpose
	approved signals.
Regression tests	Known edge fixtures do not change outputs unless version intentionally changes.

Condition	Severity
No rows yet in fresh live run	WARN
Missing artifacts	FAIL
Missing selected features	FAIL
Probability mismatch	FAIL
Calibration mismatch	FAIL
AFML mismatch	FAIL
Contract mismatch	FAIL
Unsafe selected feature	FAIL
Old rows with different run_id	IGNORE
Drift outside warning but inside hard limit	WARN
Drift outside hard limit	FAIL

27. Standard Shared Modules to Build

Module	Purpose	Used by
shared/edges/edge_layout.py	Resolve edge paths, identity, stems, artifacts, contracts.	All edge scripts and runtimes.
shared/edges/lock_selected_features_and_model_input_schema.py	Step 04B shared implementation.	All Step 04B wrappers.
shared/scoring/edge_scorer.py	Generic model/calibrator/threshold/AFML scoring.	runtime.py::score for every edge.
shared/validation/edge_contracts.py	Required API, schema, and artifact validation.	00, 05, 06, 08, 09.
shared/validation/feature_schema.py	Feature safety validation.	01, 04B, 05, 08.
shared/models/cpcv_training.py	CPCV, purge, embargo, uniqueness weights, class weights.	Step 02 and Step 03.
shared/calibration/sigmoid_afml.py	Sigmoid calibrator, AFML bet size, threshold sweep.	Step 04 and EdgeScorer.
shared/io/artifact_hash.py	Stable artifact hashing for manifests.	All export/audit gates.
shared/pipeline/edge_lifecycle.py	Optional full-stage runner/orchestrator.	Future automated pipeline.

28. Edge Generator and Future No-Assistance Workflow

The long-term professional experience should be: add an edge to config, run a generator, implement only alpha-specific logic, run standardized commands. No manual patching. No dashboard edits.

1. Add edge entry to config/edge_universe.json with status=scaffold_not_deployable.
2. Run shared edge generator to create the standard folder, contracts, runtime template, and research wrappers.
3. Implement candidate_events.py, primary_side.py, model_rows.py, and optional edge-specific features.
4. Update feature_schema.json for every produced feature.
5. Run schema and runtime self-tests.
6. Run 01 through 09 in order.
7. Only PASS edge manifests can be seen by allocator.

29. Migration Plan for Current Edges

Edge	Current status	Migration action

Edge	Current status	Migration action
XAUUSD/trend_long	Pilot edge with deployability progress/PASS manifest pattern.	Normalize to final shared-core architecture: add 04B wrapper, replace patch-style Step 06 with verifier, make runtime score use shared EdgeScorer.
XAUUSD/trend_short	Research in progress, not deployable.	Keep fail-closed. Add final shared wrappers and do not promote until full lifecycle passes.
XAUUSD/absorption_long	Step 05 PASS after 04B; promising edge.	Proceed only after shared EdgeScorer/Step 06 verifier design is implemented. Then run 07, 08, 09.
Future absorption_short	Not started.	Use generator/shared architecture from day one; do not copy trend scripts manually.

30. How to Add a New Edge

8. Define edge concept: asset, family, side, candidate event definition, label horizon, PT/SL model, and required feature families.
9. Add edge_universe entry with scaffold_not_deployable.
10. Generate scaffold from standard edge template.
11. Implement/port alpha-specific functions into runtime dependencies: feature helpers, candidate_events, primary_side, model_rows.
12. Update feature_schema.json; no feature exists without schema documentation.
13. Run 00 contract validation.
14. Run runtime feature/event/model-row self-test.
15. Export research dataset from runtime-built model rows.
16. Run 01 feature surgery, then 02 training, then 03 OOF, then 03B diagnostics.
17. Run 04 calibration and calibrated threshold sweep.
18. Run 04B selected-feature locking.
19. Run 05 artifact verifier.
20. Run 06 runtime scoring readiness verifier.
21. Run 07 runtime score smoke test.
22. Run 08 replay audit.
23. Run 09 edge manifest export.
24. Commit all scripts, contracts, manifests, and reports together. Do not commit partial deployability claims.

31. How to Add a New Asset

25. Add asset to config/asset_specs.yaml and broker_symbols.yaml.
26. Build or import raw tick data using standardized ingestion.
27. Forge standard_quote_tick bars with shared bar builder.
28. Verify asset bar manifest and hashes.
29. Build/verify FFD state if feature families require it.
30. Build/verify HMM/regime state if feature families require it.
31. Create edge packages under assets/<ASSET>/edges/.
32. Run each edge lifecycle independently.
33. Allocator picks up only PASS edge manifests; dashboard should not change.

32. Architecture Decision Record Process

This document is the implementation standard. Any deviation requires an Architecture Decision Record (ADR) under docs/decisions/. This prevents future scattered implementation from entering the codebase quietly.

```
docs/decisions/ADR-0001-title.md

# ADR-0001: Short title

Date: YYYY-MM-DD
Status: Proposed | Accepted | Superseded | Rejected
```

```
Decision owner: <name>

## Context
What problem requires deviation from the standard?

## Decision
What exact change is approved?

## Consequences
What improves, what becomes harder, and what contracts must be updated?

## Required code/document updates
- files/modules to modify
- tests to add
- migration steps

## Rollback plan
How to revert if this causes instability.
```

ADR rule

If a future AI says “just patch this one file” or “this edge is special”, it must either follow the existing standard or produce an ADR before implementation.

33. What Must Never Be Done Again

- Do not manually patch runtime.py after training. Runtime score exists from scaffold stage.
- Do not copy a Step 06 patch script from another edge and hope it is generic.
- Do not manually edit selected_features.json to fix a verifier error. Run Step 04B.
- Do not let Step 05 read feature_surgery directly as a shortcut. Step 05 verifies canonical contracts.
- Do not let dashboard calculate edge-specific features.
- Do not select final deployable thresholds using raw_probability.
- Do not deploy an edge whose selected model rows cannot be replayed through runtime.
- Do not allow old run output rows to pollute current audits; use run_id.
- Do not create hidden CSV dependencies. Every dependency must appear in a manifest/schema.
- Do not refactor all edges at once. Migrate one edge fully, prove the standard, then repeat.
- Do not weaken a fail-closed gate because an edge looks profitable.
- Do not use in-sample training predictions for deployment threshold selection.
- Do not allow edge-local wrappers to contain unique training/calibration implementations unless an ADR approves it.

34. Implementation Roadmap

Phase	Work	Exit criteria
Phase 1: Shared foundation	edge_layout, artifact hashing, feature schema validation, selected-feature lock, EdgeScorer, CPCV utilities, calibration utilities.	Shared modules have unit tests and are used by at least one edge.
Phase 2: Runtime template	Standard runtime.py from scaffold stage with fail-closed score().	New edge scaffold requires no runtime patching.
Phase 3: Normalize pilot edge	Retrofit trend_long to shared EdgeScorer and Step 06 verifier.	trend_long still passes replay and manifest export.
Phase 4: Finish absorption_long	Run Step 06 verifier, Step 07 smoke, Step 08 replay, Step 09 manifest.	absorption_long deployability PASS or documented fail.
Phase 5: Normalize trend_short	Keep research-in-progress status; add standard wrappers.	trend_short follows same lifecycle without deployability claim.
Phase 6: Allocator V2	Manifest ingestion, candidate bank, HRP, prop optimizer, live manifest export.	Allocator rejects non-PASS edges and writes live manifest.
Phase 7: Live V2	Dashboard runtime orchestration, run_id audits, execution bot signal consumption.	Demo-only live replay passes before order_send.

35. Acceptance Criteria

- A new edge can be generated and run without editing dashboard or execution bot.
- Every runtime exposes the same required API.
- Every selected feature is schema-declared, live-safe, model-input allowed, and reproduced by runtime.
- Step 02/03/04 internal MLDP logic is shared or provably identical across edges except edge-specific inputs.
- Step 04B locks selected_features.json and model_input_schema.json deterministically.
- Step 06 verifies runtime scoring readiness and never patches runtime.py.
- Step 08 proves exact replay of raw probability, calibrated probability, AFML, and threshold decision.
- Step 09 exports deployment_status=PASS only after all gates pass.
- Allocator consumes only PASS manifests.
- Dashboard and execution bot remain edge-agnostic.
- All deviations from this standard are recorded in ADRs.

36. AI/Developer Handoff Instructions

Use this instruction when handing the repo to another AI or developer:

```
You are working on an existing V2 rebuild, not a greenfield system.
Inspect the current GitHub repo first. Preserve valid MLDP/AFML logic.
Refactor toward the V2 contract-driven architecture in this document.
Do not weaken contracts for convenience.
Do not patch dashboard features.
Do not manually patch runtime.py.
Do not manually edit selected_features.json; use Step 04B.
Do not implement unique Step 02/03/04 logic inside an edge wrapper unless an ADR approves it.
Use shared utilities for repeated logic and thin edge-local wrappers for run order.
An edge is deployable only after runtime row replay, raw probability replay,
sigmoid calibration replay, AFML replay, threshold replay, and manifest export pass.
If code or a future suggestion conflicts with this document, the document wins unless superseded by an ADR.
```

37. Standard Command Runbook

Example edge lifecycle command flow. Asset/family/side arguments may be replaced by edge-local wrappers if wrappers call shared implementations.

```
# Pre-edge asset checks
python ./shared/bars/verify_asset_bar_manifest.py --asset XAUUSD
python ./shared/ffd/verify_asset_ffd_manifest.py --asset XAUUSD
python ./shared/regime/verify_asset_hmm_regime_manifest.py --asset XAUUSD

# Edge scaffold/contract checks
python ./shared/edges/verify_edge_registry.py
python ./shared/edges/verify_edge_skeleton.py --asset XAUUSD --family absorption --side long
python ./shared/edges/validate_edge_feature_schema.py --asset XAUUSD --family absorption --side long
python ./assets/XAUUSD/edges/absorption_long/runtime.py

# Research-to-live lifecycle
python ./assets/XAUUSD/edges/absorption_long/research/01_feature_surgery_clustered_mda.py
python ./assets/XAUUSD/edges/absorption_long/research/02_train_meta_model_optuna.py
python ./assets/XAUUSD/edges/absorption_long/research/03_generate_oof_predictions.py
python ./assets/XAUUSD/edges/absorption_long/research/03b_oof_threshold_sweep.py
python ./assets/XAUUSD/edges/absorption_long/research/04_fit_sigmoid_calibrator_and_thresholds.py
python ./assets/XAUUSD/edges/absorption_long/research/04b_lock_selected_features_and_model_input_schema.py
python ./assets/XAUUSD/edges/absorption_long/research/05_training_artifact_verifier.py
python ./assets/XAUUSD/edges/absorption_long/research/06_verify_runtime_scoring_ready.py
python ./assets/XAUUSD/edges/absorption_long/research/07_runtime_score_smoke_test.py
python ./assets/XAUUSD/edges/absorption_long/research/08_probability_calibration_afml_replay_audit.py
python ./assets/XAUUSD/edges/absorption_long/research/09_export_edge_manifest.py

# Allocator and live only after PASS manifests exist
python ./portfolio_allocator/scripts/build_live_manifest.py
python ./portfolio_live/dashboard.py --mode dry_run
python ./portfolio_live/execution_bot.py --mode demo_only
```

38. Appendix: Templates and Checklists

38.1 Step 06 Runtime Scoring Ready Checklist

- runtime.py imports shared EdgeScorer.
- runtime.py exposes score(model_rows=None, latest_only=False).
- EdgeScorer.from_edge_root(edge_root).validate_artifacts() passes.
- artifacts/selected_features.json exists and is locked.
- contracts/model_input_schema.json exists and matches selected_features exactly.
- Model artifact exists and can be loaded.
- Calibrator artifact exists and can be applied.
- Selected threshold exists and uses calibrated_probability.
- runtime.score(latest_only=True) returns at least one row when candidate rows exist.
- Score output contains raw_probability, calibrated_probability, afml_bet_size, selected_threshold, and approved_signal.

38.2 Edge Completion Checklist

- Asset manifests verified.
- Edge scaffold verified.
- Feature schema validated.
- Runtime feature/event/model-row self-test passed.
- Step 01 selected features produced.
- Step 02 model trained with shared CPCV logic.
- Step 03 OOF coverage complete.
- Step 04 calibration and calibrated threshold complete.
- Step 04B selected features locked.
- Step 05 training artifacts PASS.
- Step 06 runtime scoring ready PASS.
- Step 07 smoke test PASS.
- Step 08 replay audit PASS.
- Step 09 manifest deployment_status=PASS.
- Allocator has consumed PASS manifest, not research folders.

39. Scalability Must Implementation

This scalability mandate is an addendum to v1.2 and does not weaken or replace any existing contract. It makes explicit that the V2 system must scale to future assets and future edges without editing shared infrastructure, dashboard logic, allocator logic, or execution logic for each new edge.

The existing folder structure is the primary scalable identity contract. Asset identity comes from assets/<ASSET>/. Edge identity comes from assets/<ASSET>/edges/<family>_<side>/. The family is everything before the final underscore in the edge folder name. The side_name is the text after the final underscore and must be long or short. The edge_id is <ASSET>/<family>_<side>. The edge_stem is <asset_lower>_<family>_<side>. edge.yaml must confirm this derived identity; it must not become a separate hard-coded source that conflicts with the folder contract.

All shared modules, lifecycle scripts, verifiers, generators, allocator components, dashboard components, and execution components must be written to operate on this derived identity model. They must not contain enumerated lists of current edges such as trend_long, trend_short, or absorption_long, and they must not contain current-asset assumptions such as XAUUSD unless the file is explicitly an edge-local alpha module under assets/XAUUSD/edges/<edge>.

- Allowed edge-local specificity: candidate_events.py, primary_side.py, model_rows.py, and optional edge-specific feature helpers may contain alpha logic for that edge only.
- Required shared behavior: shared modules must derive asset, family, side_name, side_value, edge_id, edge_stem, and canonical artifact names from the edge root or explicit asset/family/side arguments.
- Required wrapper behavior: edge-local research wrappers must be thin and derive EDGE_ROOT from their own file path; they must call shared implementations instead of embedding generic logic.
- Forbidden scalability anti-pattern: writing separate if/elif branches for XAUUSD/trend_long, XAUUSD/trend_short, XAUUSD/absorption_long, or any future named edge inside shared modules, allocator, dashboard, or execution bot.
- Forbidden artifact anti-pattern: hard-coding model, calibrator, threshold, selected-feature, or report filenames when they can be derived from edge_stem and side_name.

- Required future-edge behavior: adding BTCUSD, USTECH, EURUSD, or any future asset must not require changes to EdgeScorer, Step 04B, Step 06, allocator ingestion, dashboard orchestration, or execution signal validation.

Derived naming contract

```
assets/<ASSET>/edges/<family>_<side>/

asset      = <ASSET> folder name, for example XAUUSD, BTCUSD, USTECH
edge_name  = <family>_<side>, for example trend_long or absorption_long
family     = edge_name before the final underscore, for example trend or absorption
side_name  = edge_name after the final underscore, long or short
side_value = +1 if side_name=long, -1 if side_name=short
edge_id    = <ASSET>/<edge_name>, for example XAUUSD/absorption_long
edge_stem  = <asset_lower>_<edge_name>, for example xauusd_absorption_long
```

Canonical artifact names must be derived, not hard-coded

```
model                  = artifacts/model/<edge_stem>_meta_model.pkl
best_params            = artifacts/model/<edge_stem>_meta_model_best_params.json
feature_surgery_output = artifacts/feature_surgery/<edge_stem>_selected_features.json
calibrator_json        = artifacts/calibration/probability_calibrator_sigmoid_side_<side_name>.json
calibrator_pkl         = artifacts/calibration/<edge_stem>_sigmoid_calibrator.pkl
calibrated_oof         = artifacts/calibration/<edge_stem>_calibrated_oof_predictions.csv
selected_threshold     = artifacts/thresholds/<edge_stem>_selected_threshold.json
threshold_sweep        = artifacts/thresholds/<edge_stem>_calibrated_threshold_sweep.csv
edge_manifest          = artifacts/edge_manifest.json
```

Scalability acceptance criteria

- A new asset folder and edge folder can be added without changing shared/scoring/edge_scorer.py.
- A new edge can run Step 04B and Step 06 using the same shared modules and only thin edge-local wrappers.
- The dashboard discovers or loads edges from the live manifest and calls runtime functions without edge-specific branches.
- The allocator consumes PASS edge manifests generically and rejects non-PASS manifests generically.
- The execution bot validates approved-signal schema and live manifest entries generically, without knowing edge alpha families.
- Any unavoidable exception to the derived naming and scalable shared-core contract requires an ADR before implementation.

Final Professional Decision

The best end-to-end design is not to keep adding edge-specific patch scripts. The correct system is a contract-driven runtime package architecture with shared reusable infrastructure, fail-closed verification gates, runtime scoring from scaffold stage, deterministic selected-feature locking, allocator-only consumption of PASS manifests, dashboard-only orchestration, and execution-only signal consumption. This document should be treated as the implementation standard. If code or future AI suggestions conflict with this standard, the standard wins unless formally updated by a new Architecture Decision Record.

Final source-of-truth rule

Contract-strict, implementation-flexible. You may improve the implementation, but you may not weaken side specialization, research/live parity, selected-feature locking, calibrated-probability thresholding, AFML replay, or manifest-gated deployment.